logoPK.png

**Politechnika Krakowska**
**im. Tadeusza Kościuszki**
Wydział Informatyki i Matematyki

logoIT.png

**Samuel Rus**

numer albumu: 151814

# Wykorzystanie wybranych metod uczenia maszynowego w predykcji efektywności sportowej zawodników Premier League.

# Predicting Premier League Player Performance Using Selected Machine Learning Methods

**praca inżynierska**

**na kierunku Matematyka stosowana**

**specjalność Analiza danych**

Praca przygotowana pod kierunkiem:
**dr Adam Sebastian Marszałek**

Kraków, 2026/27

# Streszczenie

Praca dotyczy predykcji punktacji zawodników w grze Fantasy Premier League przy użyciu metod uczenia maszynowego. Zaproponowany pełny proces przetwarzania danych, łączący dane historyczne kolejka po kolejce oraz kursy bukmacherskie przeliczone na prawdopodobieństwa. Po eksploracyjnej analizie danych oraz inżynierii cech respektującej rygor braku wycieku informacji z przyszłości, porównano trzy rodziny modeli regresyjnych: uogólnione modele liniowe, modele zespołowe oparte na drzewach decyzyjnych oraz sieci neuronowe. Zaprojektowano autorską funkcję straty oraz metryki ewaluacyjne segmentowane względem ceny rynkowej zawodnika, odzwierciedlające asymetryczną użyteczność decyzyjną błędu predykcji z perspektywy użytkownika gry. Rozważono również rozszerzenie architektury o dodatkową warstwę (dalej nazywaną meta modelem), łączącą predykcje modeli bazowych z poszanowaniem chronologicznego podziału danych, wraz z oceną wpływu takiego podejścia na jakość i wiarygodność predykcji.

# Abstract

This thesis addresses the prediction of Fantasy Premier League (FPL) player point scores using machine learning methods. A complete data pipeline is proposed, combining historical gameweek-level data (the *vaastav/Fantasy-Premier-League* repository) and bookmaker odds (*football-data.co.uk*) converted into probabilities. Following exploratory data analysis (target distribution analysis, Spearman rank correlations, PCA, k-means clustering with Kruskal–Wallis testing) and feature engineering designed to strictly avoid data leakage (pre-game features, exponential moving averages, lagged features, rolling standard deviations), three families of regression models are compared: generalized linear models (GLM), tree-based ensemble models (Random Forest, XGBoost, CatBoost), and neural networks (MLP, a 1D convolutional network, and an LSTM network). A custom loss function (`FPLLoss`) and price-segmented evaluation metrics (budget / mid / premium players) are proposed, reflecting the asymmetric decision-relevant cost of prediction errors from the perspective of an FPL player. An extension of the architecture with an additional stacking layer (a meta-model combining base-model predictions under a strict chronological data split) is also considered, together with an assessment of its impact on predictive quality and reliability.

**Keywords:** Fantasy Premier League, machine learning, regression, generalized linear models, XGBoost, CatBoost, neural networks, LSTM, loss function, stacking, sports analytics.

# Spis treści

# Wstęp

Fantasy Premier League jest jedną z najpopularniejszych gier sportowych na świecie, w której uczestnicy zarządzają wirtualną drużyną piłkarzy angielskiej Premier League, otrzymując punkty na podstawie rzeczywistych występów zawodników w danej kolejce ligowej. Gra angażuje kilkanaście milionów użytkowników w każdym sezonie, co sprawia że problem predykcji liczby punktów zdobywanych przez zawodnika w kolejnej kolejce stał się interesującym zagadnieniem badawczym, łączącym elementy analizy szeregów czasowych, regresji oraz uczenia maszynowego stosowanego do danych sportowych.

Predykcja punktacji FPL charakteryzuje się kilkoma cechami utrudniającymi zastosowanie standardowych metod regresji:

— Rozkład zmiennej celu jest silnie asymetryczny i skoncentrowany wokół zera. Znaczna część zawodników w danej kolejce nie wychodzi na boisko lub gra niewielką liczbę minut.

— Wariancja punktacji jest silnie zróżnicowana w zależności od pozycji oraz wartości rynkowej zawodnika: gracze o wysokiej cenie mają zarówno wyższą oczekiwaną liczbę punktów, jak i większą wariancję wyników, co czyni błąd predykcji dla tej grupy szczególnie kosztownym z perspektywy użytkownika podejmującego decyzje transferowe.

— Dane mają naturalną strukturę czasową, co wymusza odpowiednie podejście do konstrukcji cech opóźnionych oraz podziału zbioru na część treningową, walidacyjną i testową w sposób chronologiczny, zapobiegający wyciekowi danych z przyszłości.

Celem pracy jest zaprojektowanie, implementacja oraz porównanie kilku rodzin metod uczenia maszynowego: od uogólnionych modeli liniowych, przez modele oparte na drzewach decyzyjnych, po sieci neuronowe w zadaniu predykcji liczby punktów FPL zdobywanych przez zawodnika w pojedynczej kolejce ligowej, na podstawie danych historycznych, statystyk oczekiwanych, kursów bukmacherskich przeliczonych na prawdopodobieństwa. Wszystkie dane pochodzą z otwartych źródeł i są ujednolicane do spójnego formatu w procesie integracji. Istotnym elementem pracy jest zaprojektowanie funkcji straty oraz metryk ewaluacyjnych uwzględniających segmentację zawodników według ceny rynkowej, tak aby jakość predykcji była oceniana w sposób zbliżony do rzeczywistej użyteczności decyzyjnej dla gracza FPL.

Praca obejmuje pełny proces przetwarzania danych: od automatycznego pozyskania danych historycznych z publicznych repozytoriów i serwisów, kursy bukmacherskie, przez eksploracyjną analizę danych i inżynierię cech, po budowę, trenowanie i ewaluację modeli predykcyjnych wraz ze śledzeniem eksperymentów. Dodatkową częścią pracy, będzie wybór składu na daną kolejke Premier League.

# 1. Charakterystyka problemu

## 1.1. Zasady gry Fantasy Premier League

W grze FPL uczestnik tworzy drużynę piłkarzy Premier League, mając do dyspozycji 100 milionów funtów wirtualnego budżetu na początku sezonu. Wybór obejmuje bramkarzy, obrońców, pomocników oraz napastników. W każdej kolejce ligowej zawodnicy otrzymują punkty na podstawie zdarzeń meczowych: goli, asyst, czystych kont, kartek, interwencji bramkarskich czy statystyk wpływających na system bonusowy. Kluczową cechą systemu punktowego jest jego zależność od nominalnej pozycji zawodnika - bramkarze i obrońcy są znacznie wyżej nagradzani za czyste konto lub strzelonego gola niż gracze formacji ofensywnych, co uzasadnia podział analizy według pozycji, stosowaną w dalszych rozdziałach pracy.

Szczegółowe zestawienie zasad punktacji z uwzględnieniem podziału na pozycje boiskowe przedstawiono w Tabeli **??**.

| **Zdarzenie meczowe** | **Bramkarz** | **Obrońca** | **Pomocnik** | **Napastnik** |
|---|---|---|---|---|
| Gra do 60 minut | 1 | 1 | 1 | 1 |
| Gra co najmniej 60 minut | 2 | 2 | 2 | 2 |
| Gol | 6 | 6 | 5 | 4 |
| Asysta | 3 | 3 | 3 | 3 |
| Czyste konto (min. 60 min gry) | 4 | 4 | 1 | 0 |
| Każde 3 obronione strzały | 1 | – | – | – |
| Obrona rzutu karnego | 5 | – | – | – |
| Niewykorzystany rzut karny | -2 | -2 | -2 | -2 |
| Każde 2 stracone bramki | -1 | -1 | 0 | 0 |
| Żółta kartka | -1 | -1 | -1 | -1 |
| Czerwona kartka | -3 | -3 | -3 | -3 |
| Gol samobójczy | -2 | -2 | -2 | -2 |
| Punkty bonusowe (BPS)* | 1–3 | 1–3 | 1–3 | 1–3 |

* Punkty bonusowe przyznawane 3 najlepszym graczom wg wskaźnika BPS.

## 1.2. Sformułowanie zadania predykcyjnego

Zadanie zdefiniowano jako regresję jednowymiarową: dla zawodnika $i$ w kolejce $t$ sezonu $s$ celem jest predykcja wartości $y_{i,t,s} = \texttt{total_points}_{i,t,s} \in \mathbb{Z}_{\geqslant 0}$ na podstawie wektora cech $\mathbf{x}_{i,t,s}$ zbudowanego wyłącznie z informacji dostępnych *przed* rozpoczęciem kolejki $t$ (tzw. cechy *pre-game*), tj. bez wykorzystania statystyk z samego meczu, którego wynik jest przedmiotem predykcji. Ograniczenie to (opisane szczegółowo w rozdziale **??**) jest krytyczne dla poprawności metodologicznej – jego naruszenie prowadziłoby do wycieku danych i sztucznie zawyżonej jakości modelu w warunkach walidacji, niemożliwej do odtworzenia w rzeczywistym użyciu prognozy przed kolejką.

Dodatkowym utrudnieniem jest silna heterogeniczność zadania względem ceny rynkowej zawodnika (`value`), będącej w przybliżeniu *proxy* dla jego jakości i regularności gry. W pracy przyjęto podział zawodników na trzy segmenty cenowe – budżetowy, średni i premium – na

podstawie kwantyli rozkładu ceny (odpowiednio 50. i 95,5. percentyl), co pozwala raportować jakość predykcji oddzielnie dla każdej grupy (rozdział **??**).

# 2. Źródła i pozyskiwanie danych

## 2.1. Przegląd źródeł danych

Dane wykorzystane w pracy pochodzą z trzech niezależnych źródeł, łączonych następnie w jeden spójny zbiór przy użyciu procedur ujednolicania formatów:

— **vaastav/Fantasy-Premier-League** – publiczne repozytorium GitHub agregujące historyczne dane kolejka po kolejce oraz metadane zawodników (`players_raw.csv`) dla sezonów Premier League od 2022/23 do bieżącego sezonu 2025/26;
— **football-data.co.uk** – serwis udostępniający historyczne kursy bukmacherskie (rynek 1X2 oraz powyżej/poniżej 2,5 gola) dla poszczególnych spotkań ligowych;

## 2.2. Pozyskiwanie danych kolejka-po-kolejce (`FetchVaastav`)

Klasa `FetchVaastav` pobiera dane dla każdego sezonu i każdej z 38 kolejek osobno, łącząc je następnie z metadanymi tożsamości zawodników (`id`, imię i nazwisko, `code`) po kluczu `(element, season)`. W procesie czyszczenia danych (`_merge_and_clean`) wykonywane są następujące operacje:
— normalizacja ceny zawodnika (`value`) do jednostek właściwych grze (dzielenie przez 10);
— normalizacja popularności wyboru (`selected`) do rozkładu prawdopodobieństwa w obrębie każdej pary (sezon, kolejka), poprzez podział przez sumę w grupie;
— parsowanie i ujednolicenie znacznika czasu meczu (`kickoff_time`);
— ujednolicenie etykiety sezonu do formatu czterocyfrowego (np. `"2022-23"` → `2223`);
— usunięcie kolumn nieistotnych merytorycznie lub wprowadzonych zmianami w strukturze danych źródłowych między sezonami (m.in. cechy związane z rolą menedżera drużyny, `mng_*`);
— odrzucenie rekordów z brakami w kluczowych kolumnach identyfikacyjnych (`team`, `season`, `was_home`, `kickoff_time`).

Dodatkowo zaimplementowano metodę walidacyjną `run_tests`, realizującą automatyczny raport jakości danych (*data quality report*): kompletność liczby kolejek w sezonie (oczekiwane 38), odsetek braków danych w poszczególnych kolumnach, spójność schematu kolumn między sezonami, zakres wartości ceny zawodnika oraz wykrywanie duplikatów po kluczu głównym `(code, season, gw)`.

## 2.3. Pozyskiwanie kursów bukmacherskich (`FetchBetting`)

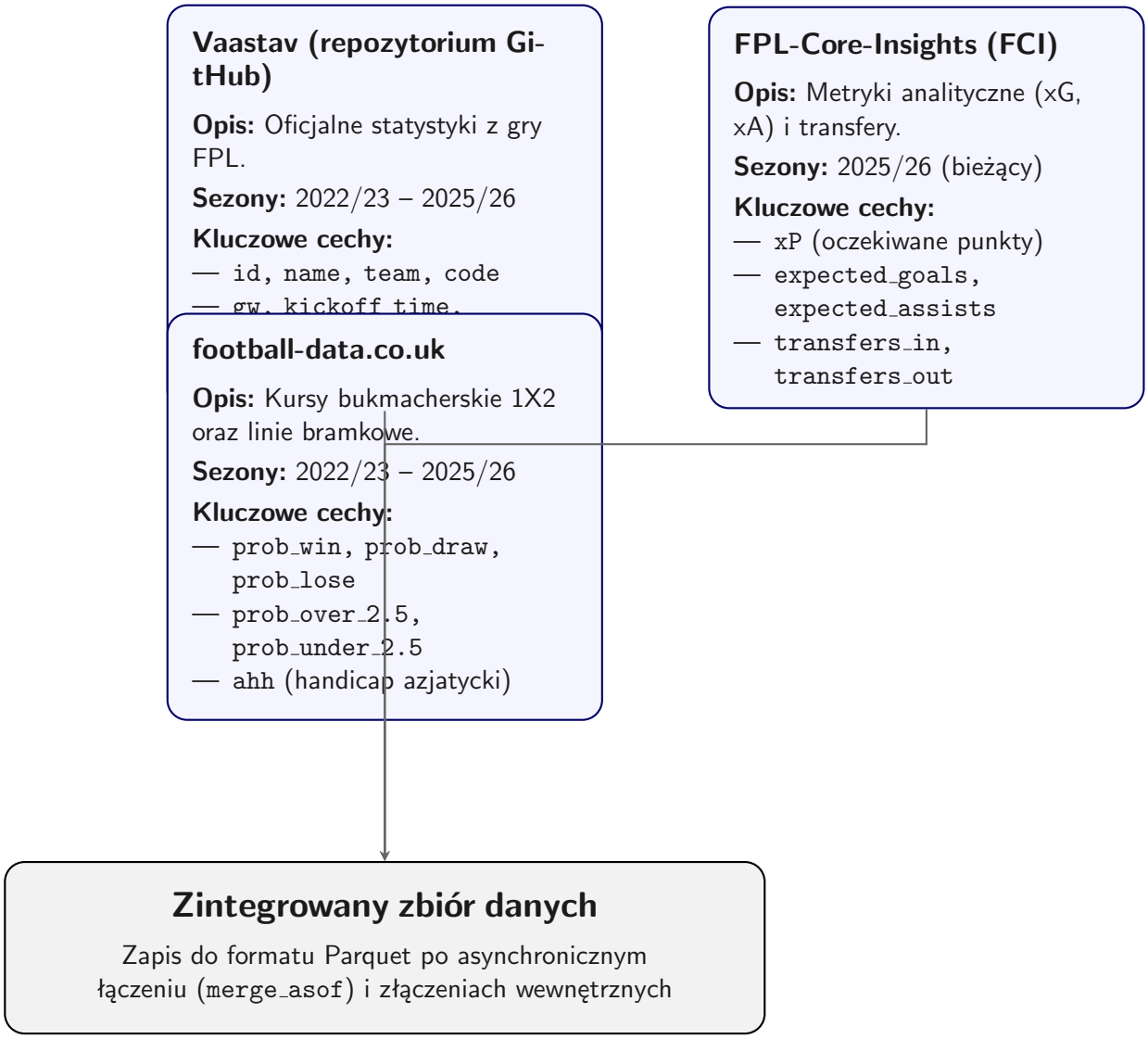
Klasa `FetchBetting` pobiera dane kursów bukmacherskich (rynek zwycięstwo gospodarzy / remis / zwycięstwo gości oraz liczba goli powyżej/poniżej 2,5) i przekształca je w dwóch krokach:

1. **Konwersja kursów na prawdopodobieństwa** – surowe prawdopodobieństwo implikowane obliczane jest jako odwrotność kursu ($\hat{p}_k = 1/\text{kurs}_k$), a następnie normalizowane w celu usunięcia marży bukmacherskiej (*overround*):

$$p_k = \frac{\hat{p}_k}{\sum_j \hat{p}_j}, \qquad \sum_k p_k = 1. \tag{2.1}$$

2. **Transformacja z formatu szerokiego do długiego** – każdy mecz (gospodarz–gość) jest rozbijany na dwa rekordy, jeden z perspektywy drużyny gospodarzy, drugi z perspektywy drużyny gości, z odpowiednio dobranymi prawdopodobieństwami zwycięstwa/remisu/porażki oraz opcjonalnym handicapem azjatyckim (`ahh`), którego znak jest odwracany dla drużyny gości.

### 2.4. Łączenie źródeł (`MergeData`)




Rysunek 2.1: Architektura zasilania modelu FPL. Wszystkie źródła przedstawiono jednolitym formatem.

Finalne połączenie trzech źródeł danych odbywa się w jednym kroku:

### 2.5. Wstępne przekształcenia

Przed właściwą analizą wykonano ujednolicenie typów danych (rzutowanie kolumn liczbowych do `float32`), zmianę nazw wybranych kolumn statystyk oczekiwanych na czytelniejsze odpowiedniki (np. `expected_goals` → `xg`) oraz normalizację statystyk ofensywnych do wartości *na 90 minut* gry ($x_{\text{per90}} = x \cdot 90/\text{minuty}$), co umożliwia porównywanie zawodników o różnym

czasie gry. Z analizy wykluczono pozycję `AM` (napastnik pomocniczy) jako niereprezentowaną w bieżącym schemacie pozycji gry.

### 2.6. Analiza rozkładu zmiennej celu

Rozkład zmiennej `total_points` scharakteryzowano za pomocą histogramu, wykresu kwantyl-kwantyl (QQ-plot) względem rozkładu normalnego oraz miar skośności i kurtozy. Analiza potwierdziła silną prawostronną skośność rozkładu oraz istotny udział wartości zerowych, wynikający z zawodników niewystępujących w danej kolejce – obserwacja ta uzasadnia wykorzystanie w dalszej części pracy rozkładów właściwych dla danych zliczeniowych (Poissona, ujemnego dwumianowego, Tweediego) w modelach GLM (rozdział **??**) oraz odpowiednio zaprojektowanej funkcji straty w modelach sieci neuronowych (rozdział **??**).

### 2.7. Analiza korelacji

Ze względu na nieliniowość i obecność wartości odstających zastosowano współczynnik korelacji rang Spearmana (nie Pearsona), obliczony osobno dla całego zbioru oraz osobno w podziale na pozycję zawodnika. Dla każdej pozycji wyznaczono ranking piętnastu (odpowiednio: jedenastu) cech najsilniej skorelowanych z `total_points`, co pozwoliło zidentyfikować różnice w istotności predyktorów między pozycjami (np. większe znaczenie cech ofensywnych xG/xA dla pomocników i napastników, a defensywnych – dla obrońców i bramkarzy).

### 2.8. Efekt gry u siebie

Zweryfikowano hipotezę o wpływie czynnika gry u siebie (`was_home`) na średnią liczbę punktów, poprzez porównanie średnich wartości `total_points` w obu grupach.

### 2.9. Redukcja wymiarowości i analiza głównych składowych (PCA)

Do standaryzowanych cech numerycznych zastosowano potok `RobustScaler` → `PCA` (skalowanie odporne na wartości odstające, istotne przy silnie skośnym rozkładzie wielu cech wydajnościowych). Przeanalizowano wariancję wyjaśnianą przez kolejne składowe (wykres *scree plot* wraz z wariancją skumulowaną) oraz wykres ładunków (*loading plot*) dwóch pierwszych składowych głównych, identyfikujący cechy o największym wkładzie w kierunki głównej zmienności danych.

### 2.10. Analiza skupień

Dla każdej pozycji osobno wykonano standaryzację cech (`RobustScaler`) i grupowanie metodą *k-średnich* (`KMeans`), z liczbą skupień $k = 3$ (interpretowaną orientacyjnie jako archetypy: gracze defensywni, uniwersalni, ofensywni w obrębie danej pozycji), a wyniki zwizualizowano w przestrzeni dwóch pierwszych składowych PCA. Istotność statystyczną różnic między skupieniami dla wybranych cech (`total_points`, `minutes`, `xg_per_90`, `xa_per_90`, `xg_conceded_per_90`, `team_xp`, `opp_xp`) oceniono testem Kruskala–Wallisa (nieparametrycznym odpowiednikiem jednoczynnikowej ANOVA, właściwym dla nienormalnych rozkładów cech), z korekcją na wielokrotne testowanie metodą Bonferroniego. Siłę efektu wyrażono współczynnikiem $\eta^2$:

$$\eta^2 = \frac{H - k + 1}{n - k}, \tag{2.2}$$

gdzie $H$ jest statystyką testu Kruskala–Wallisa, $k$ liczbą skupień, a $n$ liczbą obserwacji, przy umownej interpretacji wielkości efektu: $\eta^2 \approx 0{,}01$ – mały, 0,06 – średni, $\geqslant 0{,}14$ – duży.

# 3. Inżynieria cech

## 3.1. Zasada braku wycieku danych

Kluczowym założeniem projektowym potoku inżynierii cech jest wykorzystanie w zbiorze treningowym wyłącznie informacji dostępnych przed rozpoczęciem kolejki, której punktacja jest przedmiotem predykcji. W praktyce oznacza to, że wszystkie cechy zbudowane na podstawie statystyk meczowych (`fpl_cols`, `perf_cols`) są przesuwane w czasie (`shift`) o co najmniej jedną kolejkę, natomiast cechy dostępne *ex ante* (cena, prawdopodobieństwa bukmacherskie, rating Elo, pozycja, informacja o grze u siebie) mogą być wykorzystywane bez przesunięcia – stanowią one zbiór `pre_game_cols`.

## 3.2. Cechy oparte na historii występów zawodnika

Klasa `FeatureEngineer` generuje trzy rodziny cech historycznych, w każdym przypadku grupując dane po kluczu `(code, season)` i sortując chronologicznie po `kickoff_time`:

— **Wykładnicza średnia ruchoma (EMA)** o rozpiętości $w$:

$$\mathrm{EMA}_t(w) = (1-\alpha)\cdot \mathrm{EMA}_{t-1}(w) + \alpha \cdot x_{t-1}, \quad \alpha = \frac{2}{w+1}, \tag{3.1}$$

obliczana z przesunięciem `shift(1)`, tak aby wartość dla kolejki $t$ nie wykorzystywała statystyk z kolejki $t$;
— **Cechy opóźnione (lag)** rzędu $l$: $x_{t-l}$, dla $l \in \{1, 2, 3, 5\}$;
— **Odchylenie standardowe kroczące** w oknie $w$, również z przesunięciem o jedną kolejkę, dla $w \in \{3, 5, 8\}$, mierzące zmienność (konsystencję) formy zawodnika.

Metoda `features_integration` łączy wszystkie trzy rodziny cech (dla zadanych list rozmiarów okien) z podzbiorem cech *pre-game* oraz zmienną celu, uzupełniając ewentualne braki wynikające z niepełnej historii (początek kariery zawodnika lub sezonu) wartością zero.

## 3.3. Cechy pochodne (*feature crosses*)

Na bazie połączonych źródeł danych zbudowano dodatkowe cechy pochodne, łączące informację o formie zawodnika z kontekstem meczowym:

$$\text{transfers_trend} = \frac{\text{transfers_balance}}{\text{selected}}, \tag{3.2}$$

$$\text{prob_win_context} = \text{prob_win}, \tag{3.3}$$

$$\text{player_match_xg_expected} = \text{xg_per_90} \cdot \text{prob_win} \cdot \text{prob_over_2.5}, \tag{3.4}$$

$$\text{player_match_xa_expected} = \text{xa_per_90} \cdot \text{prob_win} \cdot \text{prob_over_2.5}, \tag{3.5}$$

$$\text{match_value_efficiency} = \frac{\text{prob_win} + 0{,}5 \cdot \text{prob_draw}}{\text{value}}, \tag{3.6}$$

$$\text{hype_vs_odds_ratio} = \frac{\text{transfers_trend}}{\text{prob_win} + 0{,}01}. \tag{3.7}$$

Wzór (**??**) został usunięty wraz z rezygnacją z ratingów Elo. Cechy `player_match_xg/xa_expected` łączą jakość ofensywną zawodnika z kontekstem trudności i charakteru meczu (prawdopodobieństwo wygranej i liczby goli), natomiast `hype_vs_odds_ratio` zestawia popularność transferową zawodnika (*sentyment* społeczności FPL) z obiektywnym prawdopodobieństwem rynkowym, umożliwiając modelowi wychwycenie rozbieżności między percepcją a oczekiwaną wydajnością.

## 3.4. Redukcja współliniowości

Po wygenerowaniu pełnego zestawu cech historycznych wykryto i usunięto cechy silnie ze sobą skorelowane (próg $|\rho| > 0{,}9$ w macierzy korelacji Pearsona, z pominięciem zmiennej celu), co ogranicza redundancję informacji i poprawia stabilność estymacji w modelach liniowych oraz interpretowalność istotności cech w modelach drzewiastych.

# 4. Modele uczenia maszynowego

W pracy porównano trzy rodziny modeli o rosnącej złożoności: uogólnione modele liniowe, modele zespołowe oparte na drzewach decyzyjnych oraz sieci neuronowe głębokie, w tym architektury dedykowane danym sekwencyjnym.

## 4.1. Uogólnione modele liniowe (GLM)

Jako model bazowy (*baseline*) zastosowano uogólniony model liniowy (bibliotekę `glum`), z regularyzacją typu *elastic net*:

$$\hat{\beta} = \arg\min_{\beta} \; -\ell(\beta; \mathbf{y}, \mathbf{X}) + \alpha \left[ \rho \|\beta\|_1 + \frac{1-\rho}{2} \|\beta\|_2^2 \right], \tag{4.1}$$

gdzie $\ell$ jest logarytmem wiarygodności rozkładu z rodziny wykładniczej. Ze względu na charakter zmiennej celu (nieujemna, silnie skośna, z nadmiarem zer) rozważono trzy rodziny rozkładów: Poissona, ujemny dwumianowy (`negative.binomial`) oraz Tweediego, z funkcją łącząca dobieraną automatycznie (`link="auto"`). Model GLM pełni w pracy rolę punktu odniesienia interpretowalnego – współczynniki $\beta$ pozwalają ocenić kierunek i siłę wpływu poszczególnych cech.

## 4.2. Modele zespołowe oparte na drzewach

Do rodziny modeli zespołowych włączono:

— **Random Forest** (`RandomForestRegressor`) jako model referencyjny o niskiej wariancji dzięki agregacji *bagging*;

— **XGBoost** (`XGBRegressor`), z funkcją celu `reg:squarederror` i metryką `rmse`, obsługą kategorii poprzez kodowanie *one-hot* realizowane w potoku `FPLDataPipe`;

— **CatBoost** (`CatBoostRegressor`), z funkcją straty `RMSE` oraz natywną obsługą cech kategorycznych (`cat_features`) bez konieczności kodowania *one-hot* – z tego względu zaimplementowano osobną wersję potoku danych (`FPLDataPipe(catboost_flag=True)`), przekazującą cechy kategoryczne jako łańcuchy znaków (`passthrough`).

Strojenie hiperparametrów obu modeli gradient boostingu realizowane jest metodą przeszukiwania losowego (`RandomizedSearchCV`) z walidacją krzyżową typu `TimeSeriesSplit` (rozdział **??**), z rozkładami hiperparametrów przedstawionymi w tabeli **??**.

## 4.3. Wielowarstwowy perceptron (MLP)

Sieć `MLP` operuje na cechach tabularycznych (zagregowanych, niesekwencyjnych) i zawiera dodatkowy, uczony mechanizm *bramkujący* selekcję cech: wektor parametrów $\mathbf{g}$ jest przepuszczany przez funkcję sigmoidalną i mnożony element-po-elemencie przez wejście,

$$\tilde{\mathbf{x}} = \mathbf{x} \odot \sigma(\mathbf{g}), \tag{4.2}$$

co pozwala sieci w trakcie treningu tłumić cechy mało informatywne. Dalsze warstwy ukryte mają strukturę `Linear` → `LayerNorm` → `LeakyReLU` → `Dropout`, z malejącym prawdopodobieństwem odrzucenia w kolejnych warstwach.

Tabela 4.1: Przestrzenie przeszukiwania hiperparametrów modeli zespołowych.

| Model | Hiperparametr | Rozkład / zakres |
|---|---|---|
| CatBoost | `iterations` | $\mathcal{U}\{150, \ldots, 500\}$ |
| | `depth` | $\mathcal{U}\{4, \ldots, 7\}$ |
| | `learning_rate` | $\mathcal{U}(0{,}01;\ 0{,}06)$ |
| | `l2_leaf_reg` | $\mathcal{U}(1{,}0;\ 9{,}0)$ |
| | `bagging_temperature` | $\mathcal{U}(0;\ 5)$ |
| | `random_strength` | $\mathcal{U}(0{,}5;\ 5{,}5)$ |
| XGBoost | `n_estimators` | $\mathcal{U}\{150, \ldots, 500\}$ |
| | `max_depth` | $\mathcal{U}\{3, \ldots, 7\}$ |
| | `subsample` | $\mathcal{U}(0{,}7;\ 1{,}0)$ |
| | `colsample_bytree` | $\mathcal{U}(0{,}6;\ 0{,}9)$ |
| | `reg_alpha` / `reg_lambda` | $\mathcal{U}(0;\ 2)$ / $\mathcal{U}(0{,}5;\ 3{,}5)$ |

## 4.4. Sieć konwolucyjna 1D (Conv1D)

Model `Conv1DRegressor` operuje na sekwencjach $T$ ostatnich kolejek zawodnika (oknie przesuwnym, opisanym w sekcji **??**), traktując wymiar czasu jako wymiar przestrzenny konwolucji jednowymiarowej. Architektura opiera się na blokach resztkowych (*residual*):

$$\mathbf{h}^{(l+1)} = \mathrm{ReLU}\big(\,\mathrm{Block}(\mathbf{h}^{(l)}) + \mathrm{Skip}(\mathbf{h}^{(l)})\,\big), \tag{4.3}$$

gdzie każdy blok składa się z dwóch warstw `Conv1d` z normalizacją wsadową (`BatchNorm1d`) i aktywacją `GELU`, a połączenie skrótowe (`Skip`) jest projekcją $1 \times 1$, gdy liczba kanałów wejściowych i wyjściowych bloku różni się. Po ostatnim bloku zastosowano globalne uśrednianie (`AdaptiveAvgPool1d`) wzdłuż wymiaru czasu, a wynik przekazywany jest do głowicy regresyjnej (`Linear` → `LayerNorm` → `LeakyReLU` → `Dropout` → `Linear`).

## 4.5. Sieć rekurencyjna LSTM

Model `LSTMRegressor` przetwarza tę samą reprezentację sekwencyjną co model konwolucyjny, poprzez warstwę projekcji wejścia, wielowarstwową sieć `LSTM` (z opcjonalną dwukierunkowością) oraz głowicę regresyjną operującą na ostatnim stanie ukrytym $h_T$ (lub konkatenacji ostatnich stanów obu kierunków w wariancie dwukierunkowym). Zarówno model konwolucyjny, jak i rekurencyjny wykorzystują ten sam mechanizm uczonej bramki selekcji cech, co model MLP.

## 4.6. Potoki danych

Dla modeli tabularycznych (GLM, modele drzewiaste, MLP) zaimplementowano klasę `FPLDataPipe`, opartą na `sklearn.ColumnTransformer`: cechy kategoryczne kodowane są metodą *one-hot* (z odrzuceniem pierwszej kategorii referencyjnej) lub przekazywane bez zmian dla CatBoost, natomiast cechy numeryczne skalowane są metodą `MinMaxScaler`, dopasowywaną wyłącznie na zbiorze treningowym.

Dla modeli sekwencyjnych (Conv1D, LSTM) zaimplementowano `SequenceDataPipe` wraz z klasą `SequenceDataset`, budującą dla każdego zawodnika i sezonu osobno okna przesuwne długości $T = 8$ kolejek: dla kolejki $t$ oknem wejściowym jest podsekwencja cech z kolejek $[t{-}T,\ t{-}1]$, a etykietą – `total_points` z kolejki $t$. Grupowanie po parze `(code, season)` zapobiega tworzeniu okien przekraczających granicę sezonu (transfer zawodnika, awans/spadek), co mogłoby wprowadzić nierealistyczną ciągłość formy.

# 5. Funkcja straty i metryki specyficzne dla FPL

## 5.1. Motywacja

Standardowy błąd średniokwadratowy (MSE) traktuje wszystkich zawodników symetrycznie, podczas gdy z perspektywy użytkownika FPL błędy predykcji dla zawodników *premium* (drodzy, kluczowi dla wyniku drużyny) są znacznie bardziej kosztowne niż dla zawodników rezerwowych, a niedoszacowanie potencjału zawodnika (*underprediction*) prowadzi do utraconych okazji transferowych w większym stopniu niż analogiczne przeszacowanie. Ponadto standardowy MSE nie uwzględnia faktu, że zawodnicy niewystępujący w danej kolejce (`minutes` $= 0$) generują z definicji zerową punktację, co bez odpowiedniej wagi zniekształca gradient w kierunku trywialnej predykcji zera.

## 5.2. Funkcja straty `FPLLoss`

Zaprojektowana funkcja straty modyfikuje bazowy błąd średniokwadratowy trzema czynnikami wagowymi:

$$\mathcal{L} = \frac{1}{N}\sum_{i=1}^{N} (y_i - \hat{y}_i)^2 \cdot w_i^{\text{cena}} \cdot w_i^{\text{asym}} \cdot (\text{minutes}_i + 0{,}1), \tag{5.1}$$

gdzie:

— **waga cenowa** $w_i^{\text{cena}}$ podbija istotność błędu dla zawodników o cenie powyżej progu `p_high` za pomocą miękkiej (różniczkowalnej) maski sigmoidalnej:

$$w_i^{\text{cena}} = 1 + (w_{\text{premium}} - 1) \cdot \sigma\big(20 \cdot (\text{price}_i - p_{\text{high}})\big); \tag{5.2}$$

— **waga asymetrii** $w_i^{\text{asym}}$ silniej karze niedoszacowanie ($y_i > \hat{y}_i$) niż przeszacowanie, poprzez funkcję `softplus` zastosowaną do różnicy błędu:

$$w_i^{\text{asym}} = 1 + (c_{\text{under}} - 1) \cdot \frac{\text{softplus}_\beta(y_i - \hat{y}_i)}{\text{softplus}_\beta(y_i - \hat{y}_i) + 1}, \quad \beta = 2; \tag{5.3}$$

— **waga minut** $(\text{minutes}_i + 0{,}1)$ obniża wpływ obserwacji zawodników niewystępujących w meczu, jednocześnie unikając zerowania gradientu (stała 0,1 zapobiega całkowitemu wyzerowaniu wagi).

Progi cenowe $p_{\text{low}}$, $p_{\text{high}}$ wyznaczane są empirycznie jako, odpowiednio, mediana (50. percentyl) i 95,5. percentyl rozkładu ceny w zbiorze treningowym, po przeskalowaniu cechy do zakresu $[0, 1]$.

## 5.3. Metryki segmentowane

Analogicznie do funkcji straty, ewaluacja modeli nie ogranicza się do zagregowanego MSE/MAE, lecz wykorzystuje autorskie metryki `CustomMSE` i `CustomMAE` (zaimplementowane jako klasy `torchmetrics.Metric`), raportujące błąd oddzielnie w trzech segmentach cenowych:

$$\text{MSE@}k = \frac{1}{|S_k|}\sum_{i \in S_k} (y_i - \hat{y}_i)^2, \qquad k \in \{\text{budget}, \text{mid}, \text{premium}\}, \tag{5.4}$$

gdzie $S_{\text{budget}} = \{i : \text{price}_i < p_{\text{low}}\}$, $S_{\text{mid}} = \{i : p_{\text{low}} \leqslant \text{price}_i < p_{\text{high}}\}$, $S_{\text{premium}} = \{i : \text{price}_i \geqslant p_{\text{high}}\}$. Metryki te są zgodne implementacyjnie zarówno z modelami tabularycznymi (indeks ceny w wymiarze cech), jak i sekwencyjnymi (indeks ceny w ostatnim kroku czasowym okna), dzięki parametrowi `model_type` (`"mlp"` / `"lstm"` / `"conv1d"`).

Dodatkowo raportowany jest skorygowany współczynnik determinacji $R^2_{\text{adj}}$:

$$R^2_{\text{adj}} = 1 - (1 - R^2) \cdot \frac{n-1}{n-p-1}, \tag{5.5}$$

gdzie $n$ jest liczbą obserwacji, a $p$ liczbą cech, co koryguje $R^2$ o efekt liczby predyktorów i umożliwia rzetelne porównanie modeli o różnej wymiarowości wejścia.

# 6. Trenowanie modeli i strojenie hiperparametrów

## 6.1. Podział danych

Podział na zbiór treningowy, walidacyjny i testowy wykonano w sposób chronologiczny, a nie losowy, co jest metodologicznie konieczne przy danych szeregów czasowych: sezony 2022/23 i 2023/24 stanowią zbiór treningowy, sezon 2024/25 – zbiór walidacyjny, a bieżący sezon 2025/26 – zbiór testowy. Taki podział odwzorowuje rzeczywisty scenariusz użycia modelu (predykcja na danych z przyszłości względem danych treningowych) i eliminuje ryzyko wycieku informacji między sezonami.

## 6.2. Skalowanie zmiennej celu

Zmienna celu jest dodatkowo skalowana metodą `MinMaxScaler` (dopasowaną na zbiorze treningowym) przed treningiem sieci neuronowych, co stabilizuje proces optymalizacji; wszystkie raportowane metryki są obliczane po odwróceniu skalowania (`inverse_transform`), a więc w oryginalnej skali punktowej.

## 6.3. Klasa `Trainer`

Uniwersalna klasa `Trainer` obsługuje trening wszystkich trzech architektur sieci neuronowych (MLP, Conv1D, LSTM) w oparciu o wspólny interfejs, obejmujący:

— pętlę treningową i walidacyjną (`train_step`, `test_step`) z akumulacją metryk segmentowanych opisanych w rozdziale **??**;

— wczesne zatrzymanie (*early stopping*) na podstawie straty walidacyjnej, z parametrem cierpliwości (`patience`) i zachowaniem najlepszego stanu wag modelu;

— harmonogram szybkości uczenia `ReduceLROnPlateau`, redukujący *learning rate* po braku poprawy straty walidacyjnej przez zadaną liczbę epok, z okresem chłodzenia (`cooldown`).

## 6.4. Strojenie hiperparametrów modeli klasycznych

Dla modeli GLM, XGBoost i CatBoost strojenie hiperparametrów realizowane jest metodą `RandomizedSearchCV` z walidacją krzyżową `TimeSeriesSplit` (zachowującą porządek czasowy w obrębie kroswalidacji, w przeciwieństwie do standardowego podziału losowego $k$-krotnego), metryką optymalizacji `neg_mean_squared_error` oraz 20 iteracjami losowania z przestrzeni hiperparametrów.

## 6.5. Śledzenie eksperymentów

Wszystkie eksperymenty (parametry, metryki epokowe, artefakty modeli) są rejestrowane w narzędziu MLflow za pośrednictwem klasy `MlflowTracker`, w tym metryki wsadowe logowane zbiorczo (`log_metrics` per epoka) w celu ograniczenia liczby wywołań sieciowych do serwera śledzącego, co miało istotne znaczenie przy uruchamianiu treningów w środowisku obliczeniowym SSPCloud/Onyxia.

# 7. Wyniki i dyskusja

*Niniejszy rozdział zostanie uzupełniony po zakończeniu pełnego cyklu eksperymentów dla wszystkich modeli opisanych w rozdziale* **??**. Zalecana struktura rozdziału:

— tabela zbiorcza porównująca modele (GLM, Random Forest, XGBoost, CatBoost, MLP, Conv1D, LSTM) pod względem MSE, MAE oraz $R^2_{\text{adj}}$, raportowanych łącznie oraz w podziale na segmenty cenowe (budget/mid/premium) i pozycje zawodników;
— krzywe uczenia (strata treningowa i walidacyjna w funkcji epoki) dla modeli sieciowych, ilustrujące działanie mechanizmu wczesnego zatrzymania;
— analiza istotności cech (*feature importance*) dla modeli drzewiastych oraz wartości nauczonej bramki selekcji cech ($\sigma(\mathbf{g})$) dla modeli sieciowych;
— analiza błędów (rezydua predykcji) w podziale na segmenty cenowe, weryfikująca skuteczność funkcji straty `FPLLoss` w ograniczaniu niedoszacowania zawodników premium;
— studium przypadku: predykcje dla wybranej kolejki bieżącego sezonu, zestawione z rzeczywistymi wynikami.

### TODO: warstwa stackingowa (meta-model)

**Status: sekcja planowana, do zaimplementowania i uzupełnienia wynikami.** Poniżej zebrano ustalenia projektowe dotyczące rozszerzenia architektury pracy o drugi poziom modelowania (*stacked generalization*), które zostanie zaimplementowane i zewaluowane przed finalnym oddaniem pracy.

**Koncepcja.** Modele bazowe opisane w rozdziale **??** (GLM, Random Forest, XGBoost, CatBoost, MLP, Conv1D, LSTM), potencjalnie trenowane na różnych wariantach funkcji straty (rozdział **??**), generują zróżnicowane, częściowo nieskorelowane wzorce błędów predykcji. Planuje się dodanie meta-modelu (docelowo prosty regularyzowany GLM/Ridge lub nieujemna kombinacja wypukła predykcji), przyjmującego jako cechy wejściowe predykcje modeli bazowych, a jako target – rzeczywistą wartość `total_points`.

**Krytyczne ryzyko: wyciek danych między poziomami.** Naiwne podejście (modele bazowe trenowane łącznie do sezonu 2025/26, meta-model testowany na tym samym sezonie) prowadzi do wycieku danych – predykcje modeli bazowych na zbiorze testowym byłyby wtedy *in-sample*, sztucznie zaniżając błąd. Przyjęta docelowa struktura trójpoziomowa:

— **trening modeli bazowych**: sezony 2022/23 + 2023/24;
— **trening meta-modelu**: predykcje *out-of-fold* zamrożonych modeli bazowych na sezonie 2024/25 jako cechy, z targetem `total_points` z tego samego sezonu;
— **ewaluacja końcowa całego potoku** (baza + meta): sezon 2025/26, przy czym modele bazowe pozostają wytrenowane wyłącznie na 2022/23–2023/24 (bez doučenia na 2024/25 lub 2025/26 przed testem).

**Rozszerzenie: walk-forward zamiast pojedynczego holdout.** Ze względu na ograniczoną liczbę sezonów treningowych dla meta-modelu (ryzyko przeuczenia przy silnie skorelowanych predykcjach modeli bazowych), rozważane jest zastąpienie pojedynczego holdout (2024/25) blokowym schematem walk-forward: trening bazy na 2022/23 → predykcje OOF na 2023/24; trening bazy na 2022/23+2023/24 → predykcje OOF na 2024/25; połączenie obu zbiorów OOF jako rozszerzony zbiór treningowy meta-modelu, z zachowaniem porządku czasowego.

**Projekt meta-modelu.**
— silna regularyzacja (Ridge / Elastic Net w `glum`) ze względu na współliniowość predykcji modeli bazowych;
— wariant alternatywny: nieujemna kombinacja wypukła wag ($w_j \geqslant 0$, $\sum_j w_j = 1$) – mniej elastyczna, ale bardziej interpretowalna i odporna na przeuczenie przy małej liczbie obserwacji;
— rozszerzenie typu *feature-weighted stacking*: dodanie do wejścia meta-modelu kontekstu (segment cenowy, pozycja, minuty z poprzedniej kolejki), umożliwiające różnicowanie wag modeli bazowych względem segmentu (np. większe zaufanie do CatBoost dla obrońców, do LSTM dla napastników).

**Baseline i istotność statystyczna.** Cały potok (baza + meta) zostanie porównany nie tylko wewnętrznie, ale także z: (i) oficjalnym `xP` (*expected points*) dostarczanym przez FPL, obecnym już w zbiorze danych, jako punktem odniesienia praktycznej użyteczności, oraz (ii) prostym uśrednieniem predykcji modeli bazowych, jako punktem odniesienia dla oceny, czy nauczony meta-model wnosi wartość ponad naiwny blend. Różnice w metrykach błędu między wariantami zostaną ocenione testem istotności statystycznej (test Diebolda–Mariano lub bootstrap na poziomie kolejek), ze względu na ograniczoną liczbę sezonów testowych.

**Uwagi dot. wdrożenia (poza zakresem ewaluacji offline).** Docelowo modele bazowe i meta-model będą wersjonowane łącznie w MLflow Model Registry (retrening meta-modelu przy każdej zmianie warstwy bazowej), a predykcje generowane cyklicznie (raz na kolejkę) w ramach zaplanowanego zadania wsadowego, bez potrzeby wnioskowania w czasie rzeczywistym.

# 8. Wybór składu — problem plecakowy

# 9. Decyzje dotyczące składu i system rekomendacji

# Podsumowanie

W pracy zaprojektowano i zaimplementowano kompletny potok predykcji punktacji zawodników Fantasy Premier League, obejmujący automatyczne pozyskiwanie i integrację danych z trzech niezależnych źródeł, eksploracyjną analizę danych z wykorzystaniem metod statystycznych odpornych na nienormalność rozkładów (korelacja Spearmana, test Kruskala–Wallisa), inżynierię cech respektującą rygor braku wycieku informacji z przyszłości oraz porównanie trzech rodzin modeli uczenia maszynowego o rosnącej złożoności. Istotnym wkładem pracy jest zaprojektowanie funkcji straty `FPLLoss` oraz towarzyszących jej metryk segmentowanych względem ceny rynkowej zawodnika, odzwierciedlających asymetryczną użyteczność decyzyjną błędu predykcji z perspektywy rzeczywistego użytkownika gry.

Do naturalnych kierunków dalszego rozwoju systemu należą: (i) budowa modelu zespołowego (*ensemble*) łączącego predykcje modeli tabularycznych i sekwencyjnych, (ii) uwzględnienie dodatkowych źródeł danych (np. statystyk zaawansowanych typu *expected threat*, danych o składach wyjściowych ogłaszanych przed meczem), (iii) przeniesienie modelu do trybu wnioskowania *online*, z cotygodniową douczaniem na najświeższych danych bieżącego sezonu, oraz (iv) rozszerzenie zadania predykcyjnego o element optymalizacji doboru składu drużyny w ramach ograniczeń budżetowych gry (zadanie programowania całkowitoliczbowego na predykcjach modelu).

# Bibliografia


[1] E. Panek, *Ekonomia matematyczna*, Akademia Ekonomiczna w Poznaniu, Poznań 2003.

[2] Vaastav Anand, *Fantasy-Premier-League* – repozytorium danych historycznych FPL, https://github.com/vaastav/Fantasy-Premier-League.

[3] *Football-Data.co.uk* – historyczne kursy bukmacherskie lig piłkarskich, https://www.football-data.co.uk.

[4] ...